



Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**

Евалуација на KumoRFM врз релациони бази на податоци

Изработил:
Христијан Гајдов

Ментор:
Соња Гивеска

Содржина

Вовед	3
Теоретски осврт и методологија	4
Парадигма на релационо длабоко учење	4
Архитектура на KumoRFM и KumoRFM-2	5
Енкодер независен од шемата (Table-Agnostic Encoder)	5
Relational Graph Transformer	5
Подобрување на пред-тренирањето преку скалирање и рана интеграција на задачите во KumoRFM-2	5
Методи на имплементација и дефинирање на задачите	6
Што е PQL и како работ?	6
Синтакса на PQL прашањата	7
Anchor и временски прозорци (Temporal Logic)	7
Конкретни дефиниции на задачи преку PQL	8
Резултати и толкување	8
Почетни чекори	8
MIMIC-III (Medical Information Mart for Intensive Care)	9
MIMIC-IV	14
Заклучок	18
Референци	19
Додаток А: Playground-от развиен во најновата верзија на Kumo платформата	20
Додаток Б: Ограничувања на бесплатната верзија на KumoRFM	24

Вовед

Во современата област на анализа на податоците, голем дел од деловните, логистичките, медицинските информации и сл. се чуваат во релациони бази на податоци структурирани во повеќе меѓуповрзани табели (релации). Традиционалниот пристап на машинско учење врз вакви комплексни релациони бази на податоци бара долготраен и макотрпен процес на рачно инженерство на својства (наречено feature engineering) и агрегација на селектираните податоци во една рамна табела. Овој процес не само што одзема многу време, туку честопати резултира со губење на клучни релациони врски и суптилни зависности помеѓу ентитетите.

Со подемот на вештачката интелигенција се отвара нова парадигма која овозможува директно учење над релационите бази на податоци во форма на граф невронски мрежи, без потреба од претходен feature engineering. Оваквите модели се наречени релациони фундаментални модели (Relational Foundation Models). Во центарот на овој проблем е платформата позната како [kumo.ai](#) и нивниот модел KumoRFM, кој користи граф невронски мрежи (GNNs) заедно со јазик за лесно дефинирање на потребните предвидувања од овие модели наречен Predictive Query Language (PQL). Овој јазик овозможува лесно и флексибилно дефинирање на предиктивни задачи директно преку синтакса слична на SQL, со автоматско генерирање на тренинг примероци соодветно усогласени со почетно време за предвидување на временски серии.

Целта на овој извештај е да изврши сеопфатна евалуација на перформансите и ефикасноста на KumoRFM моделот при решавање на различни предиктивни задачи врз комплексни релациони бази на податоци. За таа цел, во рамките на истражувањето како основа за евалуација се користат медицински податоци од различни податочни множества организирани во релации:

1. Клинички домен и интензивна нега. Евалуација на моделот во критични клинички сценарија врз баците MIMIC-III, MIMIC-IV, eICU, како и множествата ENCoDE mEasuring skiN Color (за корекција на оксиметриски девијации) и ORCHID (за трансплантација на органи). Во овој сегмент моделот се тестира на регресивни и класификациски задачи, како предвидување на морталитет кај пациенти и фреквенција на специфични дијагностички настани.
2. Системи за препорака (Recommendation Systems). Евалуација на флексибилноста на KumoRFM во предвидувањето над множествата за e-commerce и RelBench F1 за прикажување на можностите на Playground платформата на Kumo AI.

Овој труд ќе даде детален увид во способноста на KumoRFM преку анализа на различни метрики за успешност вклучувајќи точност (accuracy), прецизност (precision), одзив (recall), f1 вредност, AUROC, AUPRC и сл. за класификациони задачи, додека за регресивни задачи користени метрики се MAE (mean absolute error), MAPE (mean absolute percentage error), MSE (mean square error), RMSE (root mean square error), SMAPE (Symmetric mean absolute percentage error) и R^2 . Со помош на овие метрики ќе се добие реален поглед врз способноста KumoRFM автоматски да ги извлече и објасни зависностите и временските трендови во релационите бази на податоци, утврдувајќи ја неговата вредност како робусно решение за предиктивна аналитика во реални системи.

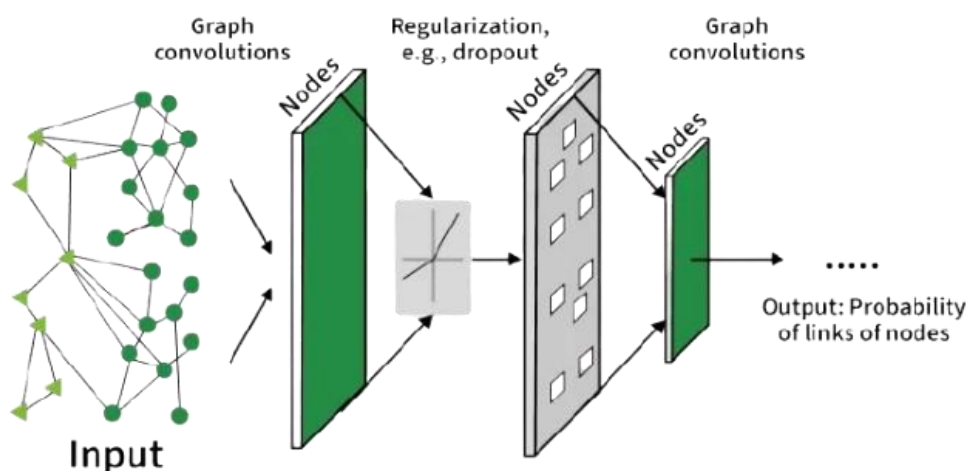
Теоретски осврт и методологија

Парадигма на релационо длабоко учење

Кито во своите трудови врши споредба помеѓу традиционалното моделирање и нивните нови Relational Foundation модели. Традиционалното моделирање на предвидувања над релациони структури како што е спомнато претходно се потпира на мануелно инженерство на својства (наречено *feature engineering*) каде податоците од поврзаните табели се агрегираат или селектираат во една единствена статична табела со која потоа се тренираат модели. Кито во овој дел од нивната евалуација вршат тренирање на повеќе модели, но главно тренираат модели базирани на одлучувачки дрвја (Gradient Boosted Decision Trees – GBDT). Овој процес се соочува со некои критични проблеми:

- Елиминирање на структурните и релационите информации, врски и зависности помеѓу примарните и надворешните клучеви, притоа губејќи важни информации од кои може да се научат многу работи за самите структури
- Подложност на мешање на тренирачките со тестирачките податоци (поим познат како *data leakage*) преку ненадејно вметнување на информации од иднината.
- Секоја нова задача за предвидување го бара истиот макотрпен процес на рачно селектирање на својства и градење на модел на новите агрегирани податоци.
- Може да одземе многу време, подлежи на грешки и бара големо искуство за најефикасно извлекување на својствата за предвидување.

Наспроти ова, релационото длабоко учење (RDL) ги претставува релациите во релационите бази на податоци директно како хетерогени временски графови. Пристапот е таков што секој ред од релациите од базата на податоци се претставува како јазол во графот, карактеристиките на редот се атрибутите на јазолот, а релациите кои се дефинирани преку надворешните клучеви се самите ребра во графот кои ги поврзуваат јазлите. На овој начин RDL го автоматизира процесот преку учење на вистинската репрезентација на базата, зачувувајќи ја комплетната топологија на базата и временскиот редослед на настаните. Ова длабоко учење се извршува со помош на граф невронски мрежи така што со помош на графот кој се гради од релациите во базата овие невронски мрежи учат соодветни својства и на различни предиктивни прашања даваат различни одговори.



Архитектура на KumoRFM и KumoRFM-2

KumoRFM претставува битна промена во справувањето со структурирани податоци бидејќи е дизајниран како модел со способност за in-context учење и fine-tuning врз основа на произволни релациони шеми. За разлика од претходните пристапи кои се ограничени на единечна табела, архитектурата на Kumo е скалабилна низ целата беза преку три клучни хиерархиски поврзани компоненти:

Енкодер независен од шемата (Table-Agnostic Encoder)

Првиот предизвик кај релационите фундаментални модели е хетерогеноста на типовите на колоните (текст, броеви, датуми...) и фактот дека секоја база има различна шема. KumoRFM го надминува овој предизвик со помош на универзален табеларен енкодер (Table-Agnostic Encoder) кој користи два слоја на трансформација:

- Серијализација на метаподатоци (Metadata Serialization). Секоја вредност во релациите се мапира заедно со името на колоната и нејзиниот тип. На пример вредноста 35.5 во колоната temperature се преведува во семантички контекст ("column: temperature, value: 35.5")
- Јазична семантичка кодификација: Преку вградени големи јазични модели (LLMs) со помал број на параметри и напредни нумерички енкодери, овие текстуални и нумерички низи се проектираат во заеднички латентен простор со фиксна димензија – d . Ова му овозможува на моделот да ги разбере значењата на колоните дури и кога се соочува со потполно нови бази на податоци за време на учење. Секој ред од секоја релација добива своја почетна векторска репрезентација односно embedding.

Relational Graph Transformer

Откако редовите од табелите ќе се трансформираат во јазли на граф со почетни ембединзи, следува Relational Graph Transformer. Наместо да се потпира на класично пренесување на пораки по ребра (Message Passing GNNs), кое често пати страда од губење на информациите по неколку скока низ јазлите (oversmoothing). Kumo го имплементира attention механизмот прилагоден за графови:

- Multi-hop расудување. Моделот извршува агрегација на информациите низ соседните табели поврзани со примарни и надворешни клучеви. На пример ембедингот на јазолот на одреден пациент (во patients) се ажурира со привлекување на информациите од неговите прегледи (admissions), кои пак истовремено повлекуваат информации од табелата за микробиолошки наоди (microbiologievents).
- Динамично доделување на тежини на релациите. Attention механизмот учи кои релации и кои специфични редови од поврзаните табели имаат најголеми предиктивна моќ за дадената задача, автоматски потиснувајќи ги нерелевантните или шумни информации.

Подобрување на пред-тренирањето преку скалирање и рана интеграција на задачите во KumoRFM-2

Во најновата архитектура, KumoRFM-2 направени се два клучни теоретски и практични чекори за подобрување на предвидувањата:

1. 4-Axis Scaling. Пред-тренирањето на KumoRFM-2 се изведува врз огромен корпус на синтетички и реални релациони бази, каде моделот се скалира и оптимизира истовремено долж четири димензии:
 - Row dimension. Справување со различен број на записи во една табела

- Column dimension. Робусност при голема димензионалност на табелите во базата.
 - Foreign key dimension. Ефикасно навигирање низ комплексни топологи и шеми.
 - Cross-sample dimension. Пренесување на знаењето помеѓу различни независни бази.
2. Рано вклучување на информациите за задачата. Ова е најголемата структурна разлика помеѓу двете верзии на KumoRFM. Кај традиционалните GNN решенија (вклучително и првиот KumoRFM) прашањето/задачата се дефинира на самиот крај, откако ќе заврши целото градење на графот. KumoRFM-2 го внесува контекстот на задачата (дефиницијата на PQL прашањето) уште во почетните слоеви на енкодерот.

Ова му овозможува на граф трансформерот да изврши поточна селекција на колоните и ребрата уште на почетокот знаејќи што точно се бара. Резултатот е драстично зголемена робусност при проблеми со висок шум, нерелевантни колони и сл.

Методи на имплементација и дефинирање на задачите

Процесот на имплементација и евалуација на платформата Kumo.ai се потпира врз автоматизирано преведување на бизнис прашањата во формални задачи за машинско учење. Наместо класично рачно инженерство на својства, Kumo го користи PQL (Predictive Query Language) – декларативен јазик развиен специфично за дефинирање на задачи за предвидување директно над податоци организирани како релациони бази на податоци.

Што е PQL и како работ?

PQL претставува јазик кој се одликува со виока апстракција, инспириран од синтаксата на SQL, кој наместо анализа и собирање на историски податоци, служи за дефинирање на предиктивни задачи за предвидување на однесувањето на ентитетите во базата.

Главната моќ на PQL лежи во тоа што истовремено ги дефинира трите клучни барања на скоро секоја предиктивна задача:

1. Основен ентитет (anchor entity). Ентитетот за кој се врши предвидувањето заедно со негов атрибут (пациент, корисник, трансакција...).
2. Целна променлива (target variable или label). Математичка операција (некаква агрегација, филтрирање или логички услов) која ни го претставува самото предвидување врз основа на настани кои се веќе во самата база.
3. Временски прозорец (Prediction Window). Временски интервал во иднината во кој се следи појавата или вредноста на целната променлива.

За разлика од традиционалните пристапи каде што податоците се ставаат во статични рамни табели. Kumo преку PQL го навигира поврзаниот релационен граф низ времето. KumoRFM ја зема PQL дефиницијата, ја анализира шемата на базата и автоматски генерира примери за тренирање со прецизно усогласени временски ознаки, со што целосно се избегнува проблемот на data leakage.

Синтакса на PQL прашањата

Генералната синтакса на едно PQL пашање ја следни следната структура:

```
PREDICT <target_expression>  
FOR <entity_specification>  
[WHERE <filters>]
```

каде што:

- PREDICT: клучен збор кој ја дефинира математичката трансформација или агрегација над табелата од интерес во релационата база.
- <target_expression>: Израз составен од функција за агрегација (COUNT, SUM, MAX, MIN, AVG...), логички филтри (WHERE соодветни услови) и дефиниција за времетраење на предиктивниот прозорец.
- FOR EACH: Ја специфицира дестинацијата на предвидувањето, односно примарниот клуч на табелата која служи како anchor за временскиот прозорец.
- WHERE: е опционален дел на прашалникот и ја има истата улога како и WHERE во SQL, така што ги селектира редиците кои го исполнуваат условот дефиниран во WHERE.

Во зависност од дефиницијата на <target_expression >, PQL поддржува три главни класи на предиктивни задачи:

- Бинарна класификација. Предвидување дали одреден настан ќе се случи или дали одреден услов ќе биде исполнет во дефинираниот временски прозорец (враќа вредности True или False).
- Повеќекласна класификација. Предвидување во која од повеќе класи некој ентитет припаѓа во зависност од податоците во базата на податоци.
- Регресија. Предвидување на нумеричка вредност (на пр. вкупна сума на трошоци, број на прегледи, времетраење во милисекунди и сл).

Anchor и временски прозорци (Temporal Logic)

Клучна карактеристика на PQL синтаксата е нејзината временска природа.

Временскиот прозорец се дефинира во рамките на изразот за предвидување преку три параметри: [relative_start, relative_end, time_unit].

Синтаксичката структура изгледа вака:

```
AGGREGATE(table.column|*, relative_start, relative_end, time_unit)
```

- relative_start и relative_end. Го означуваат почетокот и крајот на идниот предиктивен прозорец во однос на моменталната точка на набљудување (т.н. Anchor Time).
- time_unit. Временска единица која може да биде DAYS, WEEKS, MONTHS или YEARS.

Пример при користење на временската логика на KumoRFM:

Доколку PQL прашањето содржи временски параметри (0, 7, DAYS), тоа означува дека моделот ќе ги набљудува сите интеракции и настани во базата до моментот T_0 , а ќе предвидува исход кој се случува во интервалот $[T_0, T_0 + 7 \text{ дена}]$. Кога моделот се тренира, Кумо автоматски врши темпорално земање примероци на различни историски точки, поместувајќи го прозорецот наназад низ времето за да генерира валидни и робусни парови од примери и лабели.

Конкретни дефиниции на задачи преку PQL

Во рамките на овој труд имплементирани се неколку различни PQL изрази со цел да се тестира робусноста на KumoRFM врз различни предиктивни парадигми:

Задача за бинарна класификација

За проверка на способноста да предвиди екстремни однесувања кај ентитетите, дефинирана е задача која идентификува дали одреден медицински провајдер ќе завележи висок број на специфични микробиолошки настани во рок од една недела во иднината:

```
PREDICT COUNT(microbiologyevents.microevent_id, 0, 7, DAYS) > 200
FOR EACH provider.provider_id
```

При извршување на овој прашалник платформата Kumo автоматски ја идентификува шемата на графот и изведува бинарна класификација. Таа го користи почетното време, собира примери и ја пресметува дистрибуцијата на позитивни и негативни класи.

Задача за предвидување на континуирани вредности

Со цел да се евалуира прецизноста на KumoRFM при предвидување на квантитативни перформанси (како што се времиња на трки во спортски домени или временски интервали во логистика), се користи аритметичка агрегација на континуирани променливи

```
PREDICT AVG(race_results.milliseconds, 0, 30, DAYS)
FOR EACH constructor.constructor_id
```

Овде моделот ја навибира релационата шема од табелата на конструктори преку табелите за пласмани и квалификации, секогаш агрегирајќи го просечниот број на милисекунди во наредните 30 дена. KumoRFM ги користи овие релациони патеки за да изгради графови на придонеси и да генерира стабилни предвидувања без појава на аномалии во податоците.

Резултати и толкување

Во ова поглавје се презентирани и детално анализирани резултатите добиени од евалуацијата на KumoRFM моделот врз различен спектар на релациони бази на податоци. Евалуацијата беше спроведена врз бази на податоци со медицински/клинички податоци. Евалуацијата беше спроведена преку извршување на специфично дефинирани предиктивни задачи (со помош на PQL јазикот) кои опфаќаат бинарна класификација, регресија и предвидување на кумулативни количини. Успешноста на моделот е добиена преку стандардни метрики за евалуација: точност, прецизност, одзив, f1-вредност, mean absolute error, mean absolute percentage error и сл.

Почетни чекори

Потребните прашалници и задачи ќе ги извршуваме во форма на jupyter тетратки на платформата Google Collab. Сите податочни множества кои ги користиме се хостирани на платформата BigQuery. За да можеме да извршуваме задачи прво потребно е да ја инсталираме библиотеката kumoai со следната инструкција:

```
!pip install kumoai
```

Потоа за да можеме да превземеме податоци од BigQuery потребно е да се автентифицираме на следниот начин:


```
!gcloud auth application-default login
```

На крај правиме иницијализација на rfm клиентот така што го праќаме API клучот како параметар:

```
KUMO_API_KEY = os.environ.get("KUMO_API_KEY")  
rfm.init(api_key=KUMO_API_KEY)
```

Потребни библиотеки:

```
import os  
import kumoai.experimental.rfm as rfm, os  
import pandas as pd  
  
from google.cloud import bigquery
```

MIMIC-III (Medical Information Mart for Intensive Care)

Базата MIMIC-III претставува релациона база на податоци која содржи клинички податоци со голем број на пациенти, диагнози, процедури, сервиси и сл. Табелите коишто ќе ги користиме од оваа база на податоци се следните:

```
tables_dict = {  
    'patients': patients_df,  
    'admissions': admissions_df,  
    'icustays': icustays_df,  
    'diagnoses_icd': diagnoses_df,  
    'procedures_icd': procedures_df,  
    'transfers': transfers_df,  
    'services': services_df  
}
```

Така што тие се превземени на следниот начин:

```
client = bigquery.Client(project="diesel-charge-465712-f8")  
  
icustays_df = client.query("SELECT subject_id, hadm_id, icustay_id as stay_id, intime,  
outtime, los FROM `physionet-data.mimiciii_clinical.icustays`).to_dataframe()  
  
patients_df = client.query("SELECT subject_id, gender, dob FROM `physionet-  
data.mimiciii_clinical.patients").to_dataframe()  
  
admissions_df = client.query("""SELECT hadm_id, subject_id, admittance, dischtime, deathtime,  
admission_type, admission_location, insurance, religion, marital_status, ethnicity,  
discharge_location FROM `physionet-data.mimiciii_clinical.admissions`""").to_dataframe()  
  
diagnoses_df = client.query("SELECT hadm_id, subject_id, seq_num, icd9_code as icd_code FROM  
`physionet-data.mimiciii_clinical.diagnoses_icd").to_dataframe()  
  
procedures_df = client.query("SELECT hadm_id, subject_id, seq_num, icd9_code as procedure_code  
FROM `physionet-data.mimiciii_clinical.procedures_icd").to_dataframe()  
  
transfers_df = client.query("""SELECT row_id as transfer_id, hadm_id, subject_id, icustay_id  
as stay_id, eventtype, curr_careunit, intime as transfer_intime, outtime as transfer_outtime  
FROM `physionet-data.mimiciii_clinical.transfers`""").to_dataframe()
```

```
services_df = client.query("""SELECT row_id as service_id, hadm_id, subject_id, transfertime,
prev_service, curr_service FROM `physionet-data.mimiciii_clinical.services`""").to_dataframe()
```

За да продолжиме со извршување на предвидувања прво потребно е да извршиме претпроцесирање на податоците, поради тоа што датумите и времињата BigQuery ги враќа како dbdate и dbtime соодветно, што е податочен тип неподдржан од Kumo платформата:

```
def preprocess_tables_dict(tables_dict):
    for table_name, df in tables_dict.items():
        dbdate_columns = df.dtypes[df.dtypes == 'dbdate'].index.tolist()
        dbtime_columns = df.dtypes[df.dtypes == 'dbtime'].index.tolist()
        for col in dbdate_columns:
            df[col] = pd.to_datetime(df[col])

        for col in dbtime_columns:
            df[col] = pd.to_datetime(df[col], format='%H:%M:%S').dt.time

    return tables_dict
```

После тоа за полесно мануелно градење на графот дефинираме функција која сите табели ќе ги претвори во LocalTable:

```
def transform_to_rfm_table(tables_dict):
    result = {}
    for table_name, df in tables_dict.items():
        rfm_table = rfm.LocalTable(
            df = df,
            name = table_name
        ).infer_metadata()
        result[table_name] = rfm_table
    return result
```

Овие чекори се повторуваат во сите други бази кои ќе ги тестираме, па затоа овој дел ќе биде изоставен понатаму.

Следува градењето на самиот граф. За секоја табела мануелно доделуваме примарен клуч, заедно со самите врски коишто постојат меѓу примарните и надворешните клучеви. За MIMIC-III податочното множество изгледа на следниот начин:

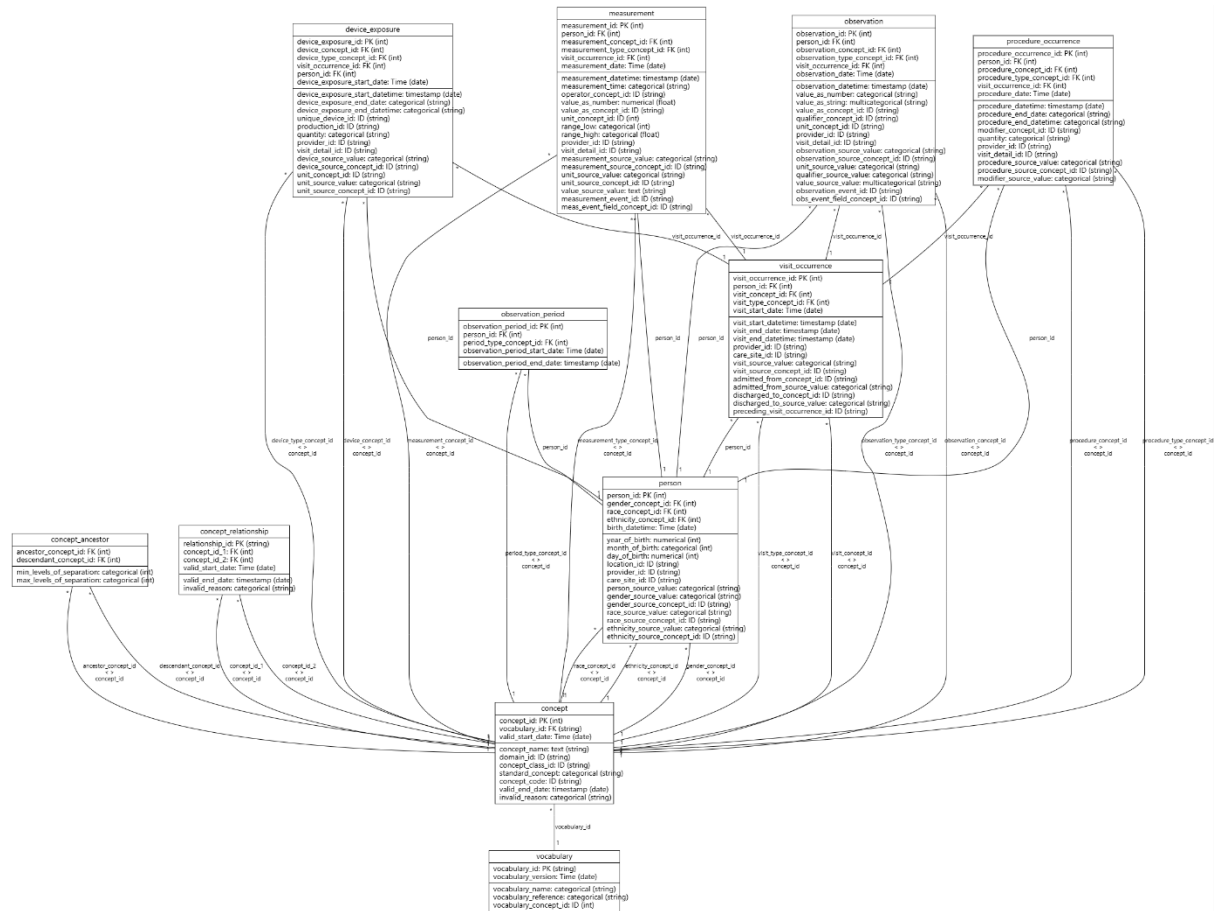
```
graph = rfm.LocalGraph(tables=tables.values())

tables['concept'].primary_key = 'concept_id'
...

graph.link(src_table='measurement', fkey='person_id',
dst_table='person')
...

graph.visualize()
```

После дефиницијата на сите овие врски добиениот граф:



Вкупно редици кои се процесираат се: 1.393.410

Откога ќе го направиме графот Relation Foundation моделот го дефинираме на следниот начин:

```
model = rfm.KumoRFM(graph=graph)
```

После овој чекор можеме да вршиме различни предиктивни задачи над дефинираниот граф. Како прва задача ќе извршиме предвидување дали одреден пациент ќе почине во рок од 48 часа за секој поединечен престој на интензивна нега. Ова претставува бинарна класификација која што дава резултат True или False.

```
PREDICT icustays.died_within_48h
```

```
FOR EACH icustays.stay_id
```

За првите пет ентитети ги добиваме следните резултати:

	ENTITY	ANCHOR_TIMESTAMP	CLASS	SCORE	PREDICTED
0	203333	2210-08-24 19:43:12	0	0.999964	True
1	203333	2210-08-24 19:43:12	1	0.000036	False
2	299036	2210-08-24 19:43:12	0	0.999855	True

	ENTITY	ANCHOR_TIMESTAMP	CLASS	SCORE	PREDICTED
3	299036	2210-08-24 19:43:12	1	0.000145	False
4	209490	2210-08-24 19:43:12	0	0.999085	True

Евалуација на самото предвидување можеме да направиме со следниот код:

```
metrics = model.evaluate(
    query,
    metrics=['acc', 'precision', 'recall', 'f1', 'mrr']
)

metrics
```

Моделот постигнува многу висока точност и прецизност при предвидувањето на смрт во рок од 48 часа, но пропушта дел од вистинските ризични случаи поради пониската вредност на recall.

ACC	F1	MRR	PRECISION	RECALL
0.977500	0.735629	0.988750	0.926872	0.66589

Исто така Кито нуди и објаснување на како моделот ги добил тие информации со поставување на параметарот `explain=True` во `model.predict()` функцијата. Во овој случај, системот автоматски генерира текстуално објаснување на резултатите, при што ги истакнува најзначајните карактеристики во податоците кои придонеле за донесување на предвидувањето, овозможувајќи полесна интерпретација и разбирање на одлуките на моделот. Изглед на `explain` карактеристиката:

```
The model predicts that the ICU stay with stay_id 35854639 has a very high risk of death within 48 hours, marking the outcome as "true" with an extremely high confidence score of 0.9998.
```

Looking into the explanation:

```
1 Global Pattern Insights (Column Analysis):
...
2 Local (Subgraph) Evidence for This Patient:
...
```

Summary Insight:

```
The prediction of death within 48 hours for this ICU stay is overwhelmingly influenced by the availability or status of the "deathtime" information in the ICU stay record, which receives the maximum importance score...
```

*оваа способност на кито платформата е ограничена при предвидувања на само еден ентитет од една релација.

Следна задача ја предвидува должуната на престој во ICU за секој престој во овој прашалник користиме `where` филтер за да ги одделиме редиците кои имаат `null` вредност за `icustays`:

```
PREDICT icustays.los
FOR EACH icustays.stay_id
WHERE icustays.los IS NOT NULL
```

Резултат:

	ENTITY	ANCHOR_TIMESTAMP	TIME	TARGET_PRED
0	13033	2187-12-17 20:54:00	2187-12-17 20:54:00	12.475324
1	13033	2187-12-17 20:54:00	2188-12-16 20:54:00	21.288597
2	13033	2187-12-17 20:54:00	2189-12-16 20:54:00	21.673878
3	13033	2187-12-17 20:54:00	2190-12-16 20:54:00	21.755569
4	13033	2187-12-17 20:54:00	2191-12-16 20:54:00	12.096891

Метриките покажуваат дека моделот има умерена точност: MAE и RMSE укажуваат на просечна грешка од неколку единици, додека $R^2 = 0.37$ значи дека моделот објаснува околу 37% од варијацијата во податоците. MAPE и SMAPE покажуваат релативно значајно процентуално отстапување помеѓу предвидените и реалните вредности.

MAE	MAPE	MSE	R^2	RMSE	SMAPE
3.446874	2.244632	82.207909	0.368378	9.066857	0.614271

Исто така, KumoRFM овозможува предвидување во повеќе идни временски рамки. Следниот прашалник го моделира бројот на посети на интензивна нега за секој пациент во следните 10 временски интервали, врз основа на историски податоци од последните 365 дена:

```
PREDICT COUNT(icustays.*, 0, 365, days)
FORECAST 10 TIMEFRAMES
FOR EACH patients.subject_id
```

Резултатите покажуваат дека моделот предвидува релативно стабилен број на ICU посети за пациентот 13033 низ идните години, со благ пораст до околу 11 посети и потоа враќање кон приближно 9. Ова укажува на умерени флуктуации без јасен силен тренд на раст или пад.

	ENTITY	ANCHOR_TIMESTAMP	TIME	TARGET_PRED
0	13033	2187-12-17 20:54:00	2187-12-17 20:54:00	9.000000
1	13033	2187-12-17 20:54:00	2188-12-16 20:54:00	10.883985
2	13033	2187-12-17 20:54:00	2189-12-16 20:54:00	11.375154

	ENTITY	ANCHOR_TIMESTAMP	TIME	TARGET_PRED
3	13033	2187-12-17 20:54:00	2190-12-16 20:54:00	11.378286
4	13033	2187-12-17 20:54:00	2191-12-16 20:54:00	9.000000

*евалуација за овој прашалник не е извршена бидејќи платформата при тестирање не дозволува негово извршување.

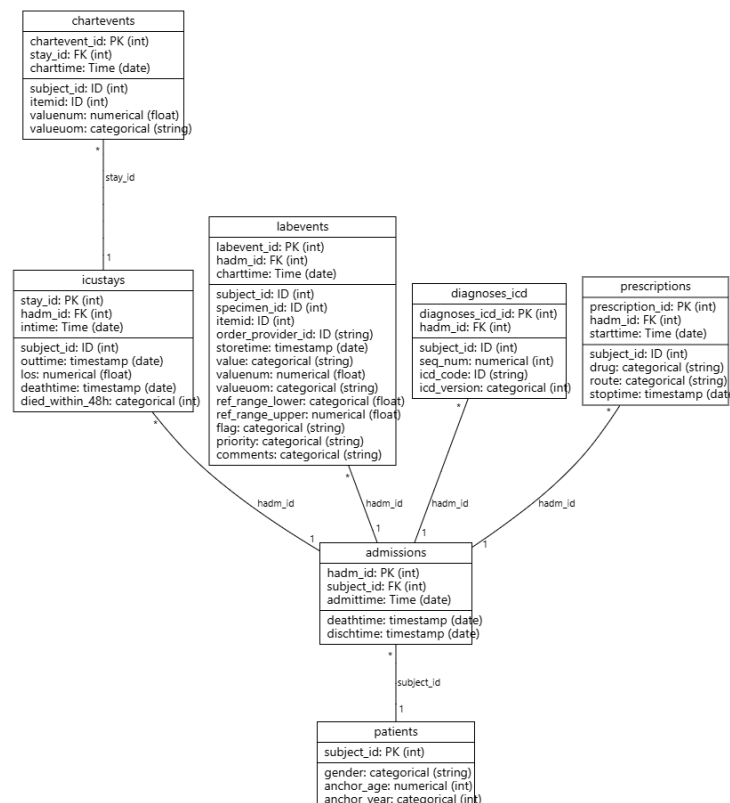
Направен е и прашалник кој го предвидува бројот на процедури во интензивна нега кои ќе ги има пациентот со subject_id=13033 во следните 10 временски рамки:

```
PREDICT COUNT(procedures_icd.*, 0, 365, days)
FORECAST 10 TIMEFRAMES
FOR patients.subject_id = 13033
```

MIMIC-IV

Базата MIMIC-IV претставува релациона база на податоци која содржи клинички податоци и претставува понова верзија на MIMIC-III. Се состои од шеми за болници, интензивна нега и изведени вредности.

Табели коишто се користат од оваа база на податоци се релациите за пациенти, кои ги опфаќаат демографските податоци, хоспитализации, престои на интензивна нега, лабораториски резултати, дијагнози, препишани терапии и клинички мерења. Процесиравме 27.635.918 (селектирани се само 1% од пациентите поради тоа што целосниот MIMIC-IV не го собира во меморијата со која што располага бесплатната верзија на Google Collab). Изглед на добиениот граф:



Како почетни предиктивни задачи ги извршуваме истите задачи за предвидување на смрт во следните 48 часа за пациент во интензивна нега и должината на престој на интензивна нега.

```
PREDICT icustays.died_within_48h
FOR EACH icustays.stay_id

PREDICT icustays.los
FOR EACH icustays.stay_id
WHERE icustays.los IS NOT NULL
```

За ова податочно множество точноста е многу помала, а останатите метрики се слични за првиот прашалник, додека за вториот прашалник MIMIC-III е посупериорен. Причината за ова се должи во тоа што во MIMIC-III користевме поголем број на табели со што многу подобро можат да се изведат зависимости помеѓу ентитетите, што доведува до подобро предвидување.

ACC	F1	MRR	PRECISION	RECALL	
0.964500	0.767487	0.982500	0.815942	0.733103	
MAE	MAPE	MSE	R ²	RMSE	SMAPE
2.434054	0.999377	30.470278	-0.076049	5.519989	0.649491

Како дополнителен прашалник извршуваме предвидување на годините на одреден пациент во зависност од релациите со коишто стапува тој:

```
PREDICT patients.anchor_age
FOR EACH patients.subject_id
WHERE patients.anchor_age IS NOT NULL
```

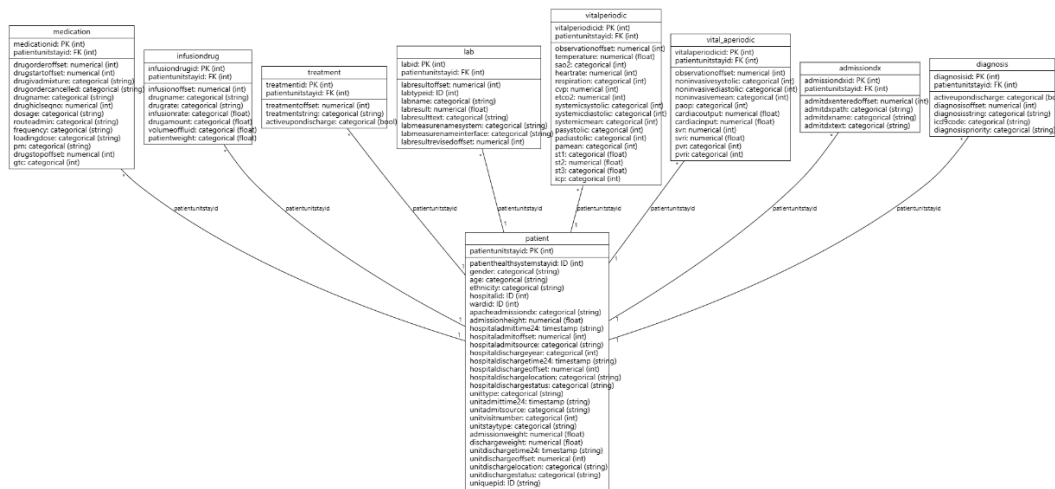
Добиените резултати не се задоволителни што значи дека потребен е поголем број на релации за да добиваме задоволителни резултати.

MAE	MAPE	MSE	R ²	RMSE	SMAPE
13.575296	0.271383	284.785492	-0.042498	16.875589	0.227866

За следните бази на податоци објаснувањата ќе бидат пократки, доколку е потребно подетално објаснување истите може да се прегледаат во прикачените материјали.

[eICU Collaborative Research Database](#)

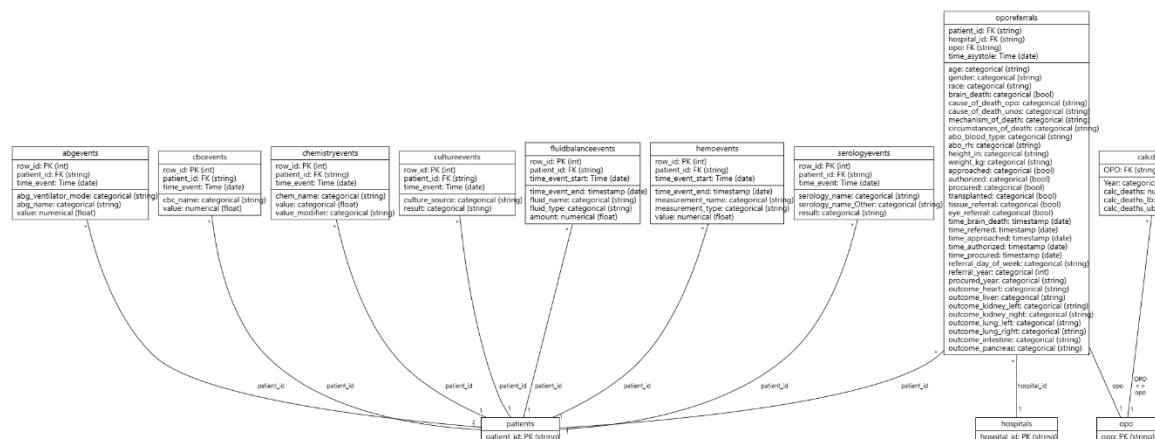
Базата eICU_CRD содржи мултицентрични податоци од единиците за интензивна нега. Врз ова множество на податоци беа извршени задачи со цел да се провери преносливоста на релационото значење кај независни медицински центри. Предвидувања кои беа извршени вклучуваат предвидување на локација каде пациентот ќе биде ослободен, дали крвниот притисок е помал од 50 како и предвидувања на самите резултати од лабораториите.



Метриките прикажаа стабилни вредности со ниска варијанса. Извлечените карактеристики покажуваат дека архитектурата на KumoRFM е отпорна на разликите во кодирањето на медицинските процедури помеѓу различните болници во eICU мрежата, благодарение на Table-Agnostic Encoder-от кој ги мапира текстуалните и нумеричките атрибути во унифициран латентен простор пред извршување на пропагацијата во графот граф.

Organ Retrieval and Collection of Health Information for Donation (ORCHID)

ORCHID базата на податоци претставува медицинско истражувачко множество на податоци што содржи клинички информации за пациенти, најчесто поврзани со болнички епизоди и лабораториски мерења. Во неа се вклучени табели како ABG (артериски крвни гасови), CBC (комплетна крвна слика) и други витални лабораториски податоци, кои се користат за анализа на состојби на пациентите. Како дополнителен медицински контекст, базата ORCHID служи за проверка на поригорозни темпорални ограничувања. Вклучени предвидувања колку OPO реферали ќе има секоја болница во наредните 7 дена, колку OPO реферали ќе се случат во еден месец, како и дали кај секој пациент ќе се појави барем еден случај во следните 24 часа каде вредноста на лактат е поголема од 4, што претставува бинарна прогноза за ризик од сепса.

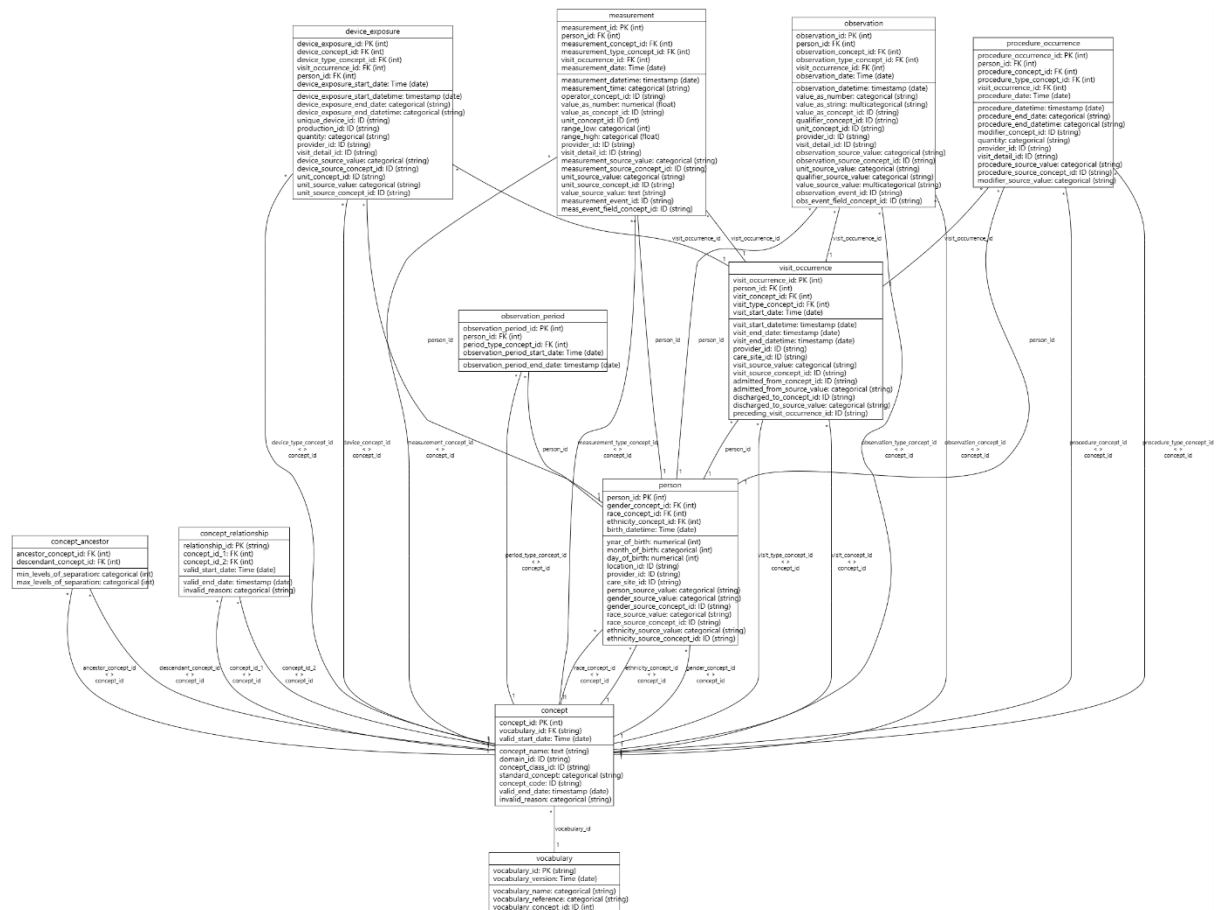


вредности за ентитетите преку ефикасно изолирање на корисниот структурен сигнал од шумот во базата.

Сепак, предвидување на финалната задача не е задоволително поради тоа што во базата нема податоци коишто ќе го исполнуваат условот, односно нема доволно податоци (екстремна реткост на настани на упатување во зададените временски прозорци од 7 и 30 дена) за моделот да направи прецизно предвидување и затоа секогаш враќа предвидување 0. Ова може да се должи и на селекцијата на `anchor_timestep`.

ENCoDE, mEasuring skin Color to correct pulse Oximetry DisparitiEs: skin tone and clinical data from a prospective trial on acute care patients.

Множеството податоци ENCoDE потекнува од истоимениот меѓународен научно-истражувачки проект (Encyclopedia of DNA Elements), чија примарна цел е да ги идентификува и мапира сите функционални елементи во човечкиот геном. Во контекст на релационите бази на податоци, овој сет претставува исклучително сложен предизвик поради неговата специфична топологија: содржи огромна димензионалност на атрибутите по табела



За евалуација на ова множество беше дефинирана специфична задача за нумеричка регресија преку PQL синтаксата, насочена кон предвидување на континуираните нумерички вредности на самите биолошки мерења:

```
PREDICT measurement.value_as_number
FOR EACH measurement.measurement_id
```

При евалуација на предиктивните перформанси за регресивната задача, анализата на дистрибуцијата на вредностите покажа дека моделот успешно ја научил сложената топологија на графот и специфичните релациски патеки помеѓу геномските елементи и нивните атрибути. Благодарение на напредната архитектура, KumoRFM демонстрира одлична стабилност во генерирањето на почетните латентни вредности за ентитетите.

Заклучок

Истражувањето спроведено во рамките на овој труд ја потврдува ефикасноста и супериорноста на парадигмата за Релационо длабоко учење имплементирана преку напредната архитектура на KumoRFM-2. Наместо традиционалното, долготрајно и макотрпно рачно инженерство на својства (feature engineering) кое често води до табели со изгубен структурен и релациски контекст, декларативниот пристап преку Predictive Query Language (PQL) овозможува брза, флексибилна и автоматизирана дефиниција на предиктивни задачи директно над поврзани граф-структури.

Преку деталната евалуација на разновиден спектар на комплексни релациони бази на податоци (MIMIC-III, MIMIC-IV, eICU_CRD, ORCHID, ENCoDE), моделот демонстрираше исклучителна способност за учење на глобалните трендови и локалните структурни патеки. Раната интеграција на контекстот на задачата во почетните слоеви на енкодерот кај KumoRFM-2 се покажува како клучен фактор за одржување висока прецизност во класификациските задачи и ниска средна апсолутна грешка во регресивните симулации.

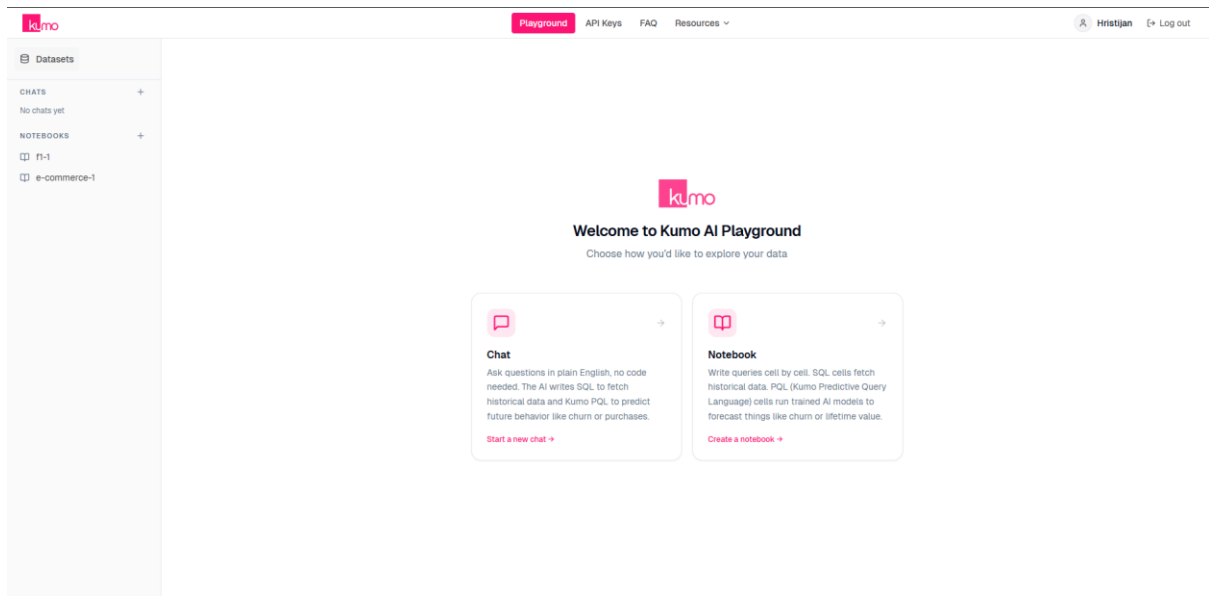
Како финален заклучок, Kumo.ai платформата и KumoRFM-2 архитектурата претставуваат револуционерен исчекор во автоматизацијата на машинското учење над релациони бази на податоци. Тие нудат моќна, скалабилна и темпорално заштитена рамка која ги трансформира суровите релациски табели во директна предиктивна вредност, со јасни насоки за понатамошни оптимизации во области со изразен дизбаланс и реткост на податоците.

Референци

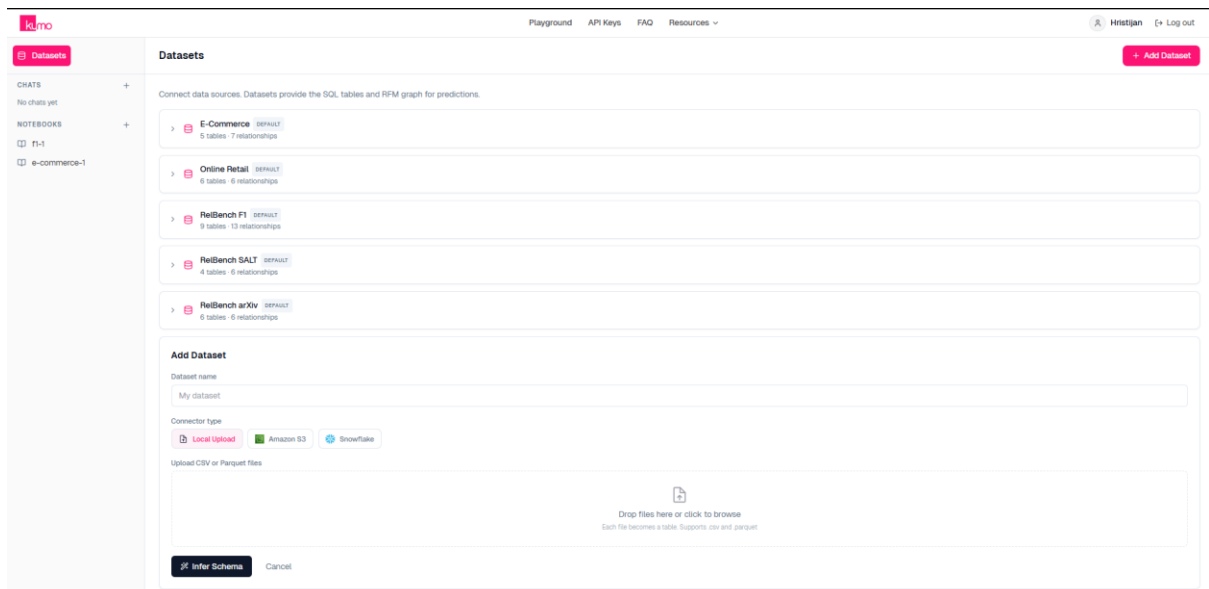
1. <https://kumo.ai>
2. <https://kumo.ai/docs/rfm/overview/>
3. <https://github.com/neo4j-contrib/northwind-neo4j>
4. <https://www.kaggle.com/datasets/grouplens/movielens-20m-dataset?resource=download>
5. <https://kumo.ai/research/>
6. <https://relbench.stanford.edu/>
7. <https://arxiv.org/pdf/2407.20060>
8. <https://www.nature.com/articles/sdata201635>
9. <https://www.nature.com/articles/s41597-022-01899-x>
10. <https://physionet.org/about/database/>
11. <https://physionet.org/content/eicu-crd/2.0/>
12. <https://physionet.org/content/encode-skin-color/1.0.0/#files-panel>
13. <https://physionet.org/content/orchid/2.1.1/>
14. <https://cloud.google.com/bigquery>
15. <https://kumo.ai/kumoRFM-2-scaling-foundation-models-for-relational-learning.pdf>

Додаток А: Playground-от развиен во најновата верзија на Kumo платформата

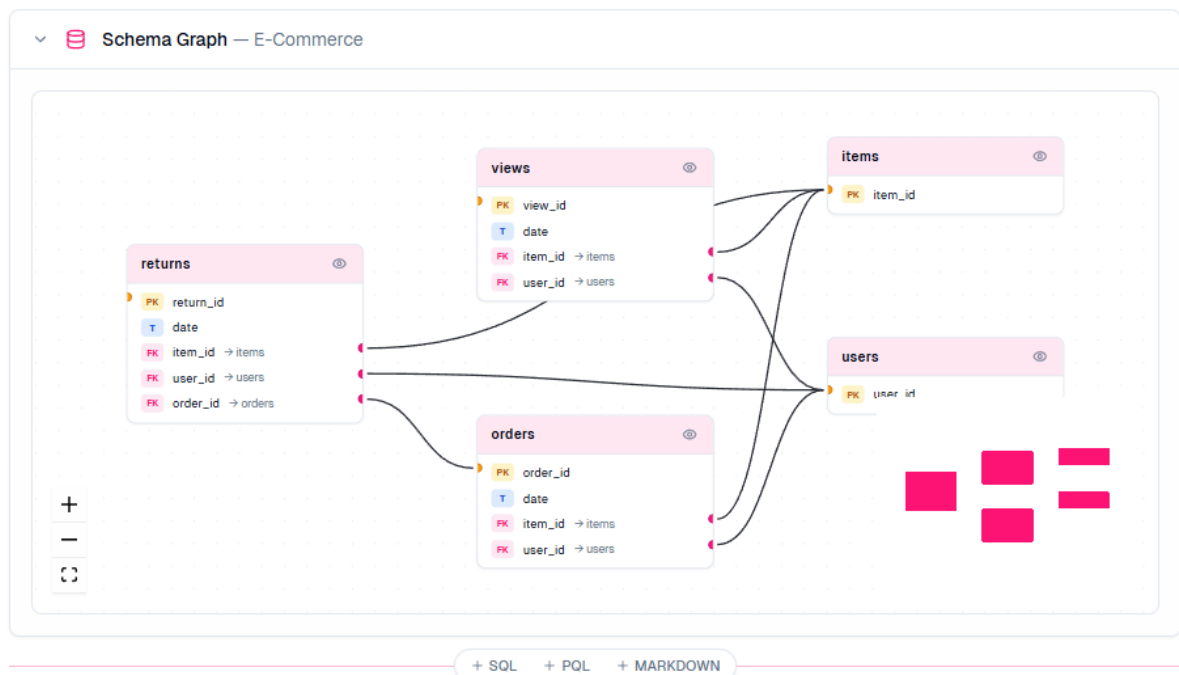
Новата верзија на Kumo платформата нуди playground, во кој можеме да поставиме свои податочни множества и на формат сличен како Jupyter тетратките да пишуваме PQL заедно со SQL прашалници.



Податочните множества се достапни со кликање на копчето Datasets, каде имаме опции да избереме од неколку предефинирани множества како и да додадеме свое податочно множество (може да се постави од Amazon S3, Snowflake и локални датотеки што е доста ограничено на максимум 100MB).



Како пример во овој труд се користени податочните множества Relbench-F1 и E-Commerce. При отварање на нова тетратка се прикажуваат ентитетите кои учествуваат во графот.



Како што може да се види можат да се додадат SQL ќелии, PQL ќелии и Markdown ќелии. SQL ќелиите едноставно ни овозможуваат да извршуваме SQL прашалници врз ентитетите и подобро да се запознаеме со самите податоци на базата.

SQL

x Clear Run

```
select * from users;
```

11463 rows x

USER_ID	FN	ACTIVE	CLUB_MEMBER_STATUS	FASHION_NEWS_FREQUENCY	AGE
0	true	true	ACTIVE	Regularly	49
1	false	false	ACTIVE	None	28
2	true	true	ACTIVE	Regularly	26
3	true	true	ACTIVE	Regularly	20
4	true	true	ACTIVE	Regularly	40

PQL ќелиите ни овозможуваат поедноставно извршување на PQL прашалници така што интуитивно се можни сите можности како explain и evaluate, исто така и при грешка во нашиот прашалник AI assist-от ни нуди решенија за нашите грешки и соодветно ни го дава решението. Со помош на оваа алатка може и текстуално да се

опишуваат предиктивните задачи и едноставно да ни генерира PQL прашалник:

PQL

Clear

Explain

Evaluate

Run

PREDICT COUNT(orders.*, 0, 30, days) > 0

FOR EACH users.user_id

5000 rows

ENTITY	ANCHOR_TIMESTAMP	TARGET_PRED	FALSE_PROB	TRUE_PROB
0	2020-09-19T00:00:00	false	0.5869964361	0.4130035639
1	2020-09-19T00:00:00	false	0.9447912574	0.055208683
2	2020-09-19T00:00:00	false	0.9931228757	0.0068771071
3	2020-09-19T00:00:00	false	0.9912548065	0.0087451823
4	2020-09-19T00:00:00	true	0.4073334336	0.592666626

Изглед на објаснувањето:

EXPLANATION

The model predicts that user 4896 is very likely to place at least one order in the next 30 days, with a high True probability of approximately 89.2%. This confident prediction is well supported by both global cohort patterns and the user's local data.

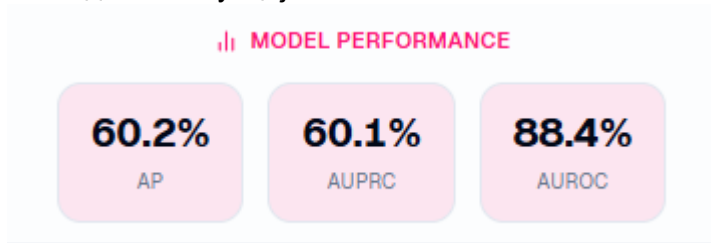
From the **global cohorts**, several points stand out:

- Users with **1+ past orders** have around 19.4% average probability of ordering again; the model here expects a much higher probability for this user, indicating strong individual signals.
- Orders with prices mostly in the **(28.31 - 30.59)** range correlate to a roughly 22.7% chance of ordering, typical of moderately priced items, suggesting likelihood is not driven solely by luxury or bargain pricing.
- Most recent order dates for users influence probability, with **orders in recent months (0 to 2 months)** showing substantially higher reorder likelihood around 35%.
- The user's cohort for **age (17)** shows a lower baseline reorder chance (about 13.4%), implying age alone would not predict high ordering—but other factors compensate.

Looking at the **local subgraph**, the model identifies several impactful factors:

- The user is **not currently active (active=False)** and has **no fashion news subscriptions (fashion_news_frequency=None)**, both moderately decreasing influence (scores ~0.01–0.05).
- However, the user is a **club member with ACTIVE status**, which carries the strongest positive effect among user attributes (score ~0.116), consistent with the global cohort showing active club members have higher reorder rates (~20.6%).
- The key drivers here are the user's **multiple recent product views dated very close to the prediction timestamp**, with several views scoring very high (e.g., scores from ~0.24 up to 0.69). This recency and frequency of views strongly increase reorder probability because the model links recent interest in items to imminent buying.
- Viewed items mostly belong to categories and features associated with moderate to high reorder likelihood, such as:
 - Item 19, a "Garment Lower body, Dark color, Solid pattern" with exceptionally high importance scores (e.g., graphical_appearance_name=1.0, perceived_color_master_name=0.75, product_type_name=0.46). These features align with product cohorts that have reorder targets in the 18–42% range or higher.
 - Other viewed items also contribute moderately via similar apparel characteristics (e.g., dark colors, solid or all-over

Изглед на евалуацијата:



* ограничување на евалуацијата е во тоа што не можеме ние да си селектираме метрики

Изглед на AI assist алатката:

The interface has a top bar with buttons: "PQL", "Clear", "Explain", "Evaluate", "Run", and a trash icon. Below is a text area with a suggested query: "# Suggested: How many views will each item have in the next 90 days? PREDICT COUNT(views.*, 0, 90, days) FOR EACH items.item_id". Below this is a section titled "AI assist" with a description "Describe what you want in PQL". It contains three input fields: "How many views will each user have in the next 3...", "How many orders will each user have in the next ...", and "How many views will each item have in the next 9...". A fourth field contains "e.g. predict spend for each user in the next 30 days" and an "Ask" button.

Исто така Кумо платформата нуди и chatbot со којшто можеме да комуницираме за подобро да ни ги објасни зависностите и самите релации кои постојат во базата на податоци така што подобро ќе се запознаеме со суштината на податоците. Chatbot-от нуди селекција на едно од нашите податочни множества и соодветно можеме да го прашуваме разни прашања кои се од наш интерес.

The interface has a top bar with "kumo", "Playground", "API Keys", "FAQ", "Resources", "Hristijan", and "Log out". On the left is a sidebar with "Datasets", "CHATS", "NOTEBOOKS", "F1-1", and "e-commerce-1". The main area has a "Welcome to Kumo AI Playground" message and a "What can I help you with today?" prompt. Below is a text input field with "Ask KumoRFM Predictive Assistant..." and buttons for "E-Commerce", "Explain", and "Evaluate". Below the input field are four suggested queries: "How many views will each user have in the next 30 days?", "How many orders will each user have in the next 90 days?", "How many views will each item have in the next 90 days?", and "How many returns will each user have in the next 180 days?".

Додаток Б: Ограничувања на бесплатната верзија на KumoRFM

При пишување на овој труд користена беше бесплатната верзија на KumoRFM.

Ограничувања коишто беа достигнати се:

1. Дозволен е максимум 1000 API повици
2. При извршување на повеќе последователни PREDICT queries, се појавува грешка: "Only one live display may be active at once"
3. Максимален број на табели во еден граф е 15.

Следната табела е преземена од официјалната документација на kumo:

	Saas	Snowflake Native	Databricks Native
Number of rows in a table	15 Billion	1 Billion	10 Billion
Number of columns in a table	2000	100	2000
Table size	2 TB	100 GB	2 TB
Row size	10 KB	10 KB	10 KB
Number of tables in a graph	12	12	12
Number of rows in a graph	30 Billion	1 Billion	30 Billion
Graph size	2 TB	200 GB	2 TB
Number of edges in a graph	30 Billion	4 Billion	30 Billion
Parallel job execution	10	1	10